Laboratório Didático de RAG Local

Manual de instalação e preparação do ambiente

Windows e GNU/Linux (Ubuntu/Debian)

Público-alvo: estudantes de graduação em Computação, Engenharia e áreas afins.
Abordagem: aprendizagem experimental, execução local, controle de versões de dependências e validação por evidências.
Infraestrutura: Python, VS Code, Docker, Elasticsearch, Kibana, Ollama e Streamlit.

Versão 1.0 – IFPR - UNIÃO DA VITÓRIA – Novembro de 2026

Apostila acompanhada de arquivos de configuração e roteiros de teste.

Disciplina Inteligência Artificial

Prof. Me. Jefferson Nogueira de Oliveira.

Índice

1 Apresentação **3**

1.1 Pré-requisitos e decisões de projeto 3

2 Arquitetura e componentes **4**

3 Organização do material fornecido **4**

4 Preparação do ambiente no Windows **5**

4.1 Passo W1 — Verificar o sistema e a virtualização 5

4.2 Passo W2 — Instalar Python e Visual Studio Code 5

4.3 Passo W3 — Instalar e verificar WSL 2 6

4.4 Passo W4 — Instalar Docker Desktop 6

4.5 Passo W5 — Preparar o limite de mapeamentos do Elasticsearch 7

4.6 Passo W6 — Instalar Ollama 7

4.7 Passo W7 — Criar o ambiente virtual e instalar bibliotecas 7

5 Preparação do ambiente GNU/Linux **8**

5.1 Passo L1 — Identificar distribuição, memória e arquitetura 8

5.2 Passo L2 — Instalar Python, venv e ferramentas auxiliares 8

5.3 Passo L3 — Instalar Docker Engine com Compose 9

5.4 Passo L4 — Testar Docker e compreender permissões 10

5.5 Passo L5 — Ajustar `vm.max_map_count` 10

5.6 Passo L6 — Instalar Ollama 11

5.7 Passo L7 — Criar e utilizar o ambiente virtual 11

6 Bibliotecas Python e isolamento do projeto **11**

6.1 Conferir importações e registrar versões 12

6.2 Configurar o interpretador no VS Code 12

7 Credenciais locais e configuração segura **12**

7.1 Criar o arquivo de credenciais 13

7.2 Verificar a ligação apenas ao computador local 13

8 Inicialização do Elasticsearch e do Kibana **13**

8.1 Passo C1 — Validar o arquivo Compose 14

8.2 Passo C2 — Iniciar somente o Elasticsearch 14

8.3 Passo C3 — Configurar a conta técnica do Kibana 14
8.4 Passo C4 — Iniciar o Kibana 14
8.5 Passo C5 — Testar o Elasticsearch sem inserir senhas no terminal 15

9 Modelos de linguagem e embeddings com Ollama 15
9.1 Passo M1 — Baixar os modelos de referência 15
9.2 Passo M2 — Testar o modelo de linguagem 16
9.3 Passo M3 — Testar geração e embeddings por Python 16

10 Verificar Streamlit e a interface web 16

11 Procedimentos de encerramento e retomada 17
11.1 Parar os serviços ao fim da aula 17
11.2 Reiniciar o modelo ou liberar memória 17

12 Segurança, permissões e boas práticas 18

13 Soluções para problemas frequentes 18

14 Checklist de preparação e relatório de evidências 20
14.1 Checklist de infraestrutura 20
14.2 Evidências para entrega ao docente 20

15 Atividades de reflexão e avaliação 21

16 Próxima etapa da disciplina 21

A Arquivo Compose de referência (`compose.yaml`) 22

B Referências técnicas oficiais 23

1 Apresentação

Este documento orienta a preparação de um laboratório para estudar *Retrieval-Augmented Generation* (RAG), ou *geração aumentada por recuperação*. Seu propósito é **didático**: cada estudante deverá compreender quais processos são executados, por que são necessários, quais serviços estão ativos e como verificar a instalação. Não se pretende entregar uma plataforma corporativa ou um serviço exposto à Internet.

Em um sistema RAG, uma pergunta é encaminhada a um mecanismo de recuperação; fragmentos relevantes são selecionados e utilizados como contexto para um modelo de linguagem. A geração da resposta deve ser acompanhada da identificação das fontes, e respostas sem evidência suficiente precisam ser tratadas como incerteza, não como fatos.

Este manual termina na **infraestrutura pronta e validada**. O desenvolvimento do indexador, da recuperação híbrida e da aplicação RAG propriamente dita constitui uma etapa posterior da disciplina. O aplicativo Streamlit fornecido neste pacote é apenas um *teste de interface*; não realiza recuperação documental.

Objetivo de aprendizagem

Ao concluir o roteiro, o estudante deverá ser capaz de: (i) explicar a arquitetura local; (ii) distinguir instalação de sistema e dependências de projeto; (iii) criar e ativar um ambiente virtual Python; (iv) configurar permissões sem desabilitar mecanismos gerais de segurança; (v) iniciar e diagnosticar serviços com Docker Compose; (vi) testar um LLM e um modelo de embeddings; e (vii) registrar evidências de reprodução do ambiente.

1.1 Pré-requisitos e decisões de projeto

A configuração de referência é uma estação com processador AMD Ryzen 5800, 32 GB de RAM e placa Radeon da série RX 500 com 4 GB de VRAM. Essa estação é apropriada para os experimentos previstos **desde que os modelos sejam pequenos e a execução não dependa de aceleração por GPU**. As placas dessa geração não devem ser presumidas compatíveis com a pilha ROCm atual. Assim, o projeto prioriza a CPU, evitando instalação de drivers experimentais e configuração de GPU nos contêineres.

Recurso	Uso planejado
CPU Ryzen 5800	Inferência local de LLM pequeno e cálculo de embeddings.
32 GB RAM	Sistema, IDE, contêineres e modelos de IA.
GPU Radeon 4 GB	Sem exigência de CUDA, ROCm ou aceleração gráfica.
Disco	Espaço para Docker, imagens, modelos e materiais (recomenda-se reservar pelo menos 25 GB livres).
Rede	Necessária para baixar os instaladores, imagens e modelos; após o preparo, os experimentos locais podem operar sem Internet.

Atenção: segurança ou configuração

Não confunda **RAG local** com **treinamento de um modelo**. Neste laboratório, utilizaremos modelos previamente treinados para produzir embeddings e gerar respostas; os documentos serão armazenados e consultados, não usados para reajustar os pesos do LLM. Não se deve prometer que resultados em CPU serão rápidos: o tempo de resposta depende

da máquina, do modelo e do tamanho do contexto.

2 Arquitetura e componentes

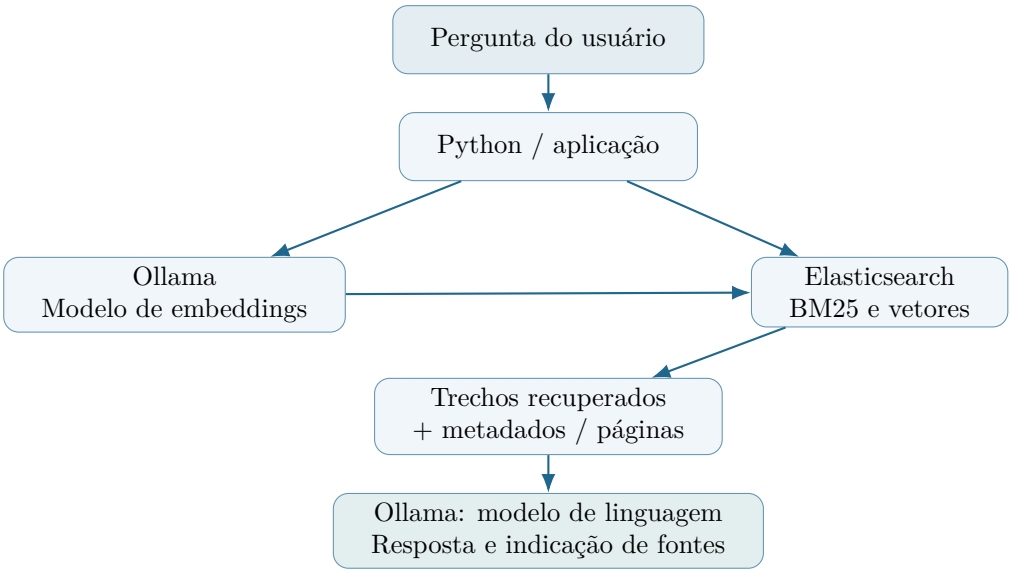


Figura 1: Arquitetura conceitual das próximas aulas. O primeiro encontro prepara os componentes, sem ainda implementar todo o fluxo RAG.

Componente	Responsabilidade
Python 3	Scripts de configuração, extração, consulta, avaliação e interface.
VS Code	Edição, terminal integrado e depuração de projetos Python.
Docker/Compose	Inicialização reproduzível dos serviços Elasticsearch e Kibana.
Elasticsearch 9.5.6	Indexação e buscas; inclui suporte a vetores.
Kibana 9.5.6	Inspeção, diagnóstico e consultas manuais.
Ollama	Serviço de inferência no próprio computador.
`qwen3:1.7b`	Pequeno modelo multilíngue para os primeiros testes.
`embeddinggemma`	Geração local de vetores numéricos de textos.
Streamlit	Interface de validação e, em atividades posteriores, interface de perguntas.

A versão **9.5.6** da pilha Elastic foi escolhida para manter compatibilidade com os experimentos anteriores da disciplina. A versão do Elasticsearch e a do Kibana devem ser idênticas dentro deste roteiro. O código Python utiliza intervalos de versões para reduzir a rigidez de instalação, mas a reprodução rigorosa requer registrar as versões efetivamente instaladas em cada estação.

3 Organização do material fornecido

Antes de prosseguir, extraia o pacote do laboratório para uma pasta **sem privilégios de administrador**, por exemplo `C:\RAG -Local` (Windows) ou `~/rag-local` (GNU/Linux). Os exemplos a seguir usam a pasta-raiz do pacote, que contém:

```
rag_local_laboratorio/
```

```
|-- compose.yaml
|-- requirements.txt
|-- .env.example
|-- .gitignore
|-- app_teste.py
|-- scripts/
|   |-- criar_env.py
|   |-- configurar_kibana.py
|   |-- validar_ambiente.py
|   `-- testar_modelos.py
|-- documentos/
|-- manual_rag_local_windows_linux.tex
`-- README.md
```

Os arquivos executáveis foram concebidos para funcionar tanto no PowerShell como no Bash. O arquivo `compose.yaml` sobe Elasticsearch e Kibana; `scripts/criar_env.py` gera senhas aleatórias; `scripts/configurar_kibana.py` configura a conta técnica do Kibana; e os testes verificam dependências e comunicação com os serviços.

Atenção: segurança ou configuração

Se o projeto for colocado sob controle de versão, o arquivo `.env` **não deverá ser enviado a repositórios**. Ele contém senhas e será criado localmente. Da mesma forma, a pasta `.venv` não deve ser versionada. O arquivo `.gitignore` fornecido já contém essas exclusões.

4 Preparação do ambiente no Windows

Este roteiro considera Windows 10/11 de 64 bits com atualizações compatíveis, PowerShell e permissões para instalar aplicações. Em computadores institucionais, políticas de domínio podem impedir ações específicas: nesses casos, a equipe de TI deverá habilitar o recurso, sem retirar as políticas de segurança.

4.1 Passo W1 — Verificar o sistema e a virtualização

Abra **PowerShell** e execute:

```
winver
systeminfo
Get-ComputerInfo | Select-Object WindowsProductName, OsVersion
```

`winver` abre uma janela com a versão do Windows. Em `systeminfo`, verifique a presença dos requisitos de virtualização. Também é possível consultar **Gerenciador de Tarefas → Desempenho → CPU → Virtualização**. O Docker com WSL 2 exige virtualização por hardware habilitada na BIOS/UEFI e componentes compatíveis do Windows.

Se a virtualização estiver desabilitada, solicite ao responsável pelo laboratório que habilite o recurso AMD-V/SVM na BIOS/UEFI. Essa etapa não pode ser corrigida apenas por um `pip install`.

4.2 Passo W2 — Instalar Python e Visual Studio Code

1. Acesse o sítio oficial do Python: python.org/downloads/windows. Para uma turma uniforme, o docente deve adotar a **mesma versão de Python 3** em todas as estações (por exemplo, Python 3.12, quando disponível e aprovado pelo setor de TI).

2. Durante a instalação, confira a disponibilidade do comando `py` e, quando aplicável, a inclusão do Python no `PATH`. Reabra o terminal após a instalação.
3. Instale o VS Code pelo site code.visualstudio.com e adicione a extensão oficial **Python** da Microsoft.

Teste:

```
py --version
py -m pip --version
code --version
```

Resultado esperado: Python e pip apresentam números de versão; `code -version` mostra a versão do editor, caso o instalador tenha configurado o comando. A inexistência do comando `code` não impede iniciar o programa pelo menu Iniciar.

4.3 Passo W3 — Instalar e verificar WSL 2

Abra um **PowerShell com privilégios administrativos**, apenas nesta etapa, e use:

```
wsl --status
wsl --install
wsl --update
```

A instalação pode solicitar reinicialização. Caso o WSL já esteja instalado, não é necessário repetir a instalação. Para conferir as distribuições presentes:

```
wsl --list --verbose
```

Em condições normais, o Docker Desktop pode usar o próprio backend WSL 2 sem que o estudante utilize uma distribuição Linux diretamente. As configurações podem variar conforme a edição e a versão do Windows; consulte a documentação oficial em Instalar o WSL.

Atenção: segurança ou configuração

Não desabilite o antivírus, o Firewall do Windows, o Controle de Conta de Usuário (UAC) ou políticas de domínio para fazer o WSL funcionar. Se a instalação estiver bloqueada, registre o erro e procure o administrador. Em máquinas com políticas institucionais, comandos administrativos podem não estar disponíveis para alunos.

4.4 Passo W4 — Instalar Docker Desktop

Baixe pelo endereço oficial Docker Desktop para Windows. Utilize o backend de **contêineres Linux sobre WSL 2**. A modalidade de instalação por usuário, quando oferecida, pode evitar a necessidade de privilégios elevados para o instalador; habilitar recursos do Windows ainda pode exigir a equipe de TI.

Inicie o Docker Desktop e confirme que o mecanismo está operacional. No PowerShell comum:

```
docker --version
docker compose version
docker info
docker run --rm hello-world
```

No primeiro uso, a imagem `hello-world` será baixada da Internet. O texto de sucesso indica que o cliente consegue se comunicar com o servidor Docker.

Como verificar

Abra o Docker Desktop e confirme a condição de execução. Acesso a `docker -version` *sozinho* não comprova que o mecanismo está funcionando; `docker info` e `docker run -rm hello-world` são testes mais completos.

4.5 Passo W5 — Preparar o limite de mapeamentos do Elasticsearch

O Elasticsearch em Docker utiliza o parâmetro de kernel `vm.max_map_count`. Em ambientes atuais, recomenda-se verificar se o valor é pelo menos **1048576**. No backend WSL 2 do Docker Desktop, quando o comando estiver disponível, consulte:

```
wsl -d docker-desktop -u root -- sysctl vm.max_map_count
```

Se o valor for inferior ao recomendado e o Elasticsearch informar falha de inicialização, solicite a alteração ao responsável pelo laboratório:

```
wsl -d docker-desktop -u root -- sysctl -w vm.max_map_count=1048576
```

A mudança acima pode não persistir entre reinicializações do backend. A configuração permanente depende da versão do WSL e da política adotada na instituição; deve ser realizada pela equipe de TI conforme a documentação da Elastic. Não é necessário editar configurações do kernel indiscriminadamente quando o ambiente já está adequado.

4.6 Passo W6 — Instalar Ollama

Instale a versão para Windows pela página ollama.com/download. O instalador normalmente configura o comando no `PATH` e executa o serviço local em segundo plano. Feche e reabra o terminal:

```
ollama --version
ollama list
```

O serviço deve responder localmente pela porta **11434**. O comando `ollama list` pode apresentar uma lista vazia antes do download dos modelos. A instalação do Ollama é separada da instalação de suas bibliotecas Python.

4.7 Passo W7 — Criar o ambiente virtual e instalar bibliotecas

Abra um **PowerShell comum** e navegue até a pasta extraída do projeto (substitua o caminho pela localização real):

```
cd C:\RAG-Local\rag_local_laboratorio
py -m venv .venv
```

Atenção à sintaxe: o comando termina após `.venv`. Não concatene duas instruções, por exemplo `py -m venv .venvpy -m venv .venv` — isso gera erros de argumentos não reconhecidos.

Se a ativação de scripts estiver autorizada:

```
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Se o PowerShell bloquear o `Activate.ps1`, verifique primeiro as políticas:

```
Get-ExecutionPolicy -List
```

Quando não houver bloqueio imposto pela instituição, uma alteração **temporária, restrita à sessão atual** pode ser feita com:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\.venv\Scripts\Activate.ps1
```

Alternativa recomendada, sem alterar política alguma: não é obrigatório ativar um `venv`. Use diretamente seu interpretador:

```
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe --version
```

O material adota essa alternativa quando um comando deve funcionar mesmo sem ativação. Ao concluir uma sessão com o ambiente ativado, use `deactivate`.

Atenção: segurança ou configuração

Não execute `Set-ExecutionPolicy Unrestricted -Scope LocalMachine`, nem desabilite políticas do PowerShell. `RemoteSigned -Scope Process` se limita ao terminal atual e não altera políticas de grupo; mesmo assim, deve ser usado somente quando permitido. Caso a instituição bloqueie a alteração, utilize o executável do `venv` diretamente.

5 Preparação do ambiente GNU/Linux

As instruções desta seção foram escritas para **Ubuntu e Debian** com arquitetura x86_64, gerenciador APT e serviço `systemd`. Outras distribuições (Fedora, openSUSE, Arch e derivadas) exigem comandos de instalação específicos; os princípios do laboratório continuam os mesmos, mas as linhas APT não devem ser executadas nelas.

5.1 Passo L1 — Identificar distribuição, memória e arquitetura

Abra um terminal e execute:

```
cat /etc/os-release
uname -m
free -h
lscpu
```

O resultado esperado de `uname -m` é `x86_64`. Registre a versão da distribuição e certifique-se de que há espaço em disco:

```
df -h ~
```

5.2 Passo L2 — Instalar Python, venv e ferramentas auxiliares

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip \
  python3-dev build-essential curl ca-certificates git
python3 --version
```

```
python3 -m pip --version
```

O pacote `python3-venv` é necessário para criar o ambiente virtual em várias instalações Ubuntu/Debian. Instalar bibliotecas da disciplina diretamente com `sudo pip` deve ser evitado, pois modifica o Python gerenciado pela distribuição e pode causar conflitos.

Para o VS Code, use o pacote publicado pela Microsoft em code.visualstudio.com/download, seguindo o procedimento adequado à distribuição e à política institucional. Depois, instale a extensão oficial Python. Verifique com `code -version`, caso a aplicação tenha registrado o comando.

5.3 Passo L3 — Instalar Docker Engine com Compose

O procedimento oficial envolve adicionar o repositório da Docker e instalar os pacotes do motor e do plugin Compose. **Escolha apenas uma** das alternativas abaixo.

Alternativa A: Ubuntu

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF2
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF2

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Alternativa B: Debian

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF2
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF2
```

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Se houver uma instalação anterior (por exemplo, pacotes `docker.io` ou `docker-compose` de outra origem), a equipe de TI deverá avaliar conflitos antes da instalação. Não é recomendável removê-los automaticamente em computadores compartilhados. As instruções podem mudar com novas versões da distribuição; valide o repositório na documentação oficial.

5.4 Passo L4 — Testar Docker e compreender permissões

```
sudo systemctl enable --now docker
sudo docker version
sudo docker compose version
sudo docker run --rm hello-world
```

O procedimento acima não exige incluir o estudante no grupo `docker`. Por padrão, o *daemon* do Docker é um serviço privilegiado. **Adicionar um usuário ao grupo `docker` concede capacidades equivalentes às de administrador/root**; portanto, essa alteração não deve ser feita automaticamente nos computadores de uso compartilhado.

Se a política institucional exigir Docker sem `sudo`, a decisão de concessão de privilégios deve partir da equipe de TI. Alternativas incluem execução com `sudo` autorizada apenas ao docente ou avaliação do Docker em modo *rootless*. No roteiro GNU/Linux, os comandos Compose aparecem com `sudo` por esse motivo.

Atenção: segurança ou configuração

Não execute `sudo chmod 666 /var/run/docker.sock` e não altere permissões globais do socket do Docker para resolver *permission denied*. Isso abriria acesso ao daemon a usuários que não deveriam administrá-lo. Documente o problema e solicite uma configuração institucional apropriada.

5.5 Passo L5 — Ajustar `vm.max_map_count`

Consulte a configuração atual:

```
sysctl vm.max_map_count
```

Se ela estiver abaixo de `1048576`, um administrador pode aplicar:

```
sudo sysctl -w vm.max_map_count=1048576
```

Para persistir após reinicialização, caso permitido pela administração da máquina:

```
printf 'vm.max_map_count=1048576\n' | \
  sudo tee /etc/sysctl.d/99-elasticsearch-lab.conf
sudo sysctl --system
sysctl vm.max_map_count
```

A alteração pertence ao sistema operacional, não ao `venv`. Em laboratórios administrados centralmente, recomenda-se executá-la uma única vez durante a preparação institucional.

5.6 Passo L6 — Instalar Ollama

A instalação simplificada disponibilizada pelo projeto é:

```
curl -fsSL https://ollama.com/install.sh -o ollama-install.sh
less ollama-install.sh
sh ollama-install.sh
```

O download e a leitura prévia do script são práticas prudentes em vez da execução imediata de conteúdo remoto. Confirme o domínio de origem e as regras de instalação da instituição. O instalador poderá solicitar privilégios administrativos e, em instalações com systemd, criar o serviço `ollama`.

```
ollama --version
sudo systemctl status ollama
ollama list
```

Se o serviço não estiver iniciado, use `sudo systemctl start ollama`; se o comando `systemctl` não estiver disponível, consulte o método de execução manual descrito na documentação oficial. Não instale componentes ROCm adicionais apenas por existir uma Radeon RX 500.

5.7 Passo L7 — Criar e utilizar o ambiente virtual

Entre no diretório do projeto (altere o caminho conforme sua extração):

```
cd ~/rag-local/rag_local_laboratorio
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

A ativação altera temporariamente variáveis do terminal. O interpretador utilizado passa a ser o da pasta virtual, em `.venv/bin/python`. A instalação de bibliotecas é feita sem `sudo`. Se desejar dispensar ativação, execute:

```
./.venv/bin/python -m pip install -r requirements.txt
./.venv/bin/python --version
```

Como verificar

Com o ambiente ativado, execute `python -c "import sys; print(sys.executable)"`. O caminho apresentado deve pertencer à pasta `.venv`. O comando `deactivate` retorna ao interpretador global.

6 Bibliotecas Python e isolamento do projeto

O arquivo `requirements.txt` acompanha o projeto e descreve as dependências. Seu conteúdo é reproduzido a seguir para transparência:

```
elasticsearch>=9.5,<10
ollama>=0.5,<1
streamlit>=1.45,<3
PyMuPDF>=1.25,<2
pandas>=2.2,<4
python-dotenv>=1,<2
requests>=2.32,<3
```

```
scikit-learn>=1.6,<2
pytest>=8,<10
```

Biblioteca	Finalidade pedagógica
`elasticsearch`	Cliente Python para consultar, indexar e inspecionar dados do Elasticsearch.
`ollama`	Cliente Python para solicitar geração de respostas e embeddings.
`streamlit`	Criação de interfaces simples para experimentos interativos.
`PyMuPDF`	Extração de texto de documentos PDF (importado em Python como `fitz`).
`pandas`	Organização de resultados, métricas e tabelas.
`python-dotenv`	Leitura de parâmetros e segredos locais em arquivo `.env`.
`requests`	Testes HTTP e configuração administrativa controlada.
`scikit-learn`	Métricas, vetorização e experimentos comparativos.
`pytest`	Testes automatizados nas próximas aulas.

6.1 Conferir importações e registrar versões

Windows (PowerShell):

```
.\.venv\Scripts\python.exe -m pip list
.\.venv\Scripts\python.exe -m pip freeze > versoes-instaladas.txt
```

GNU/Linux (Bash):

```
./.venv/bin/python -m pip list
./.venv/bin/python -m pip freeze > versoes-instaladas.txt
```

O arquivo `versoes-instaladas.txt` é uma evidência de reprodutibilidade da estação, mas também registra dependências transitivas; ele **não deve substituir sem revisão** o arquivo de dependências do projeto mantido pelo docente. As versões instaladas podem variar conforme a data de instalação, o sistema e a resolução de pacotes.

6.2 Configurar o interpretador no VS Code

Abra a pasta do projeto no VS Code. Use **Ctrl+Shift+P → Python: Select Interpreter**. Selecione `.venv\Scripts \python .exe` no Windows ou `.venv/bin/python` em GNU/Linux. No terminal integrado, confirme:

```
python -c "import sys; print(sys.executable)"
```

Se a saída apontar para outro Python, configure novamente o interpretador ou execute explicitamente o interpretador do ambiente virtual. O fato de o VS Code identificar uma extensão Python *não* garante, sozinho, que o projeto use o `venv` correto.

7 Credenciais locais e configuração segura

O Elasticsearch será iniciado com autenticação habilitada e HTTP sem TLS **apenas no contexto de loopback**. Tanto Elasticsearch como Kibana serão publicados explicitamente no endereço

`127.0.0.1`, de modo que as portas **9200 e 5601** não sejam intencionalmente expostas à rede do campus. Não se deve reutilizar essa configuração como infraestrutura de produção.

7.1 Criar o arquivo de credenciais

O script `scripts/criar_env.py` gera senhas aleatórias para os usuários técnicos e uma chave de criptografia de sessão do Kibana. Ele não sobrescreve um arquivo existente.

Windows:

```
.\.venv\Scripts\python.exe scripts\criar_env.py
```

GNU/Linux:

```
./.venv/bin/python scripts/criar_env.py
```

O arquivo gerado terá esta estrutura, com valores aleatórios exclusivos em cada estação:

```
STACK_VERSION=9.5.6
ELASTIC_PASSWORD=<senha-gerada-localmente>
KIBANA_SYSTEM_PASSWORD=<senha-gerada-localmente>
KIBANA_ENCRYPTION_KEY=<chave-gerada-localmente>
```

Os textos entre sinais `<` e `>` acima são *apenas exemplos ilustrativos*, não instruções para digitação. O valor real é criado automaticamente pelo script. O usuário `elastic` será utilizado para login e tarefas administrativas no laboratório. A conta `kibana_system` existe para comunicação interna do Kibana com o Elasticsearch e **não deve ser utilizada para entrar na interface do Kibana**.

Atenção: segurança ou configuração

A associação de `.env` a um volume persistente do Elasticsearch cria uma responsabilidade: **não regenere as senhas com os contêineres já inicializados**. Se o `.env` mudar, o volume poderá manter credenciais antigas, gerando falhas de autenticação. Guarde o arquivo de modo adequado às regras institucionais e não o anexe a relatórios ou capturas de tela.

7.2 Verificar a ligação apenas ao computador local

O Compose fornecido declara explicitamente:

```
ports:
  - "127.0.0.1:9200:9200"
```

Para o Kibana, a configuração correspondente utiliza `127.0.0.1:5601:5601`. O endereço `127.0.0.1` designa o próprio computador. Assim, `http://127.0.0.1:9200` consultará o Elasticsearch *da estação do estudante*, e não o de uma máquina remota. Em computadores compartilhados, a equipe de TI deve ainda considerar o isolamento entre contas e a persistência de volumes locais.

8 Inicialização do Elasticsearch e do Kibana

8.1 Passo C1 — Validar o arquivo Compose

Antes de iniciar contêineres, navegue até a raiz do projeto e verifique:

```
docker compose config --quiet
```

No GNU/Linux, utilize `sudo docker compose config -quiet` se essa for a política do computador. A ausência de mensagens de erro sugere que a sintaxe e as variáveis da configuração foram interpretadas corretamente. O comando exige a existência do arquivo `.env` criado na etapa anterior.

8.2 Passo C2 — Iniciar somente o Elasticsearch

Windows:

```
docker compose up -d elasticsearch
docker compose ps
```

GNU/Linux:

```
sudo docker compose up -d elasticsearch
sudo docker compose ps
```

A primeira execução baixará a imagem oficial definida no Compose. Aguarde até o serviço atingir a condição saudável. Para inspecionar logs:

```
docker compose logs --tail=80 elasticsearch
```

No GNU/Linux, acrescente `sudo` ao início do comando. A imagem exige recursos de kernel adequados e memória disponível; se o serviço reiniciar continuamente, consulte a seção de diagnóstico.

8.3 Passo C3 — Configurar a conta técnica do Kibana

O Kibana precisa autenticar-se no Elasticsearch. Para isso, o script fornecido utiliza uma chamada autenticada à API de segurança do Elasticsearch e configura a senha do usuário `kibana_system` para o valor registrado no `.env`.

Windows:

```
.\.venv\Scripts\python.exe scripts\configurar_kibana.py
```

GNU/Linux:

```
./.venv/bin/python scripts/configurar_kibana.py
```

Resultado esperado: `Senha interna do Kibana configurada com sucesso.` O script interrompe com uma mensagem de erro se a autenticação falhar. Não é necessário expor a senha em comandos de terminal ou salvar dados de autenticação no histórico do PowerShell/Bash.

8.4 Passo C4 — Iniciar o Kibana

O serviço Kibana está em um perfil opcional chamado `ui`. Isso permite executar aulas de recuperação textual mesmo sem iniciar a interface web, reduzindo o consumo de memória.

Windows:

```
docker compose --profile ui up -d
docker compose --profile ui ps
```

GNU/Linux:

```
sudo docker compose --profile ui up -d
sudo docker compose --profile ui ps
```

Em seguida, abra no navegador:

`http://127.0.0.1:5601`

O login utiliza o usuário `elastic` e a senha do campo `ELASTIC_PASSWORD` do arquivo `.env`. Não use `kibana_system` como usuário humano. O serviço pode necessitar de algum tempo adicional para carregar após o Elasticsearch estar disponível.

Como verificar

Uma sequência bem-sucedida apresenta: (1) `docker compose ps` com Elasticsearch ativo; (2) script de configuração da conta técnica concluído; (3) Kibana abrindo a página de autenticação; e (4) login realizado com `elastic`. Capturas de tela não devem mostrar o arquivo de credenciais.

8.5 Passo C5 — Testar o Elasticsearch sem inserir senhas no terminal

O script `scripts/validar_ambiente.py` consulta `http://127.0.0.1:9200` com autenticação, obtendo a senha exclusivamente de `.env`. Assim, evita-se colocar uma senha no histórico de comandos. Execute:

Windows:

```
.\.venv\Scripts\python.exe scripts\validar_ambiente.py
```

GNU/Linux:

```
./.venv/bin/python scripts/validar_ambiente.py
```

A saída indicará a presença das bibliotecas Python e o estado do Elasticsearch, Ollama e Kibana. Se ainda não tiver baixado os modelos do Ollama, a listagem de modelos pode aparecer vazia, sem invalidar o teste de conectividade.

9 Modelos de linguagem e embeddings com Ollama

O Ollama é um serviço de inferência: não é o mesmo programa que o cliente Python `ollama`. A instalação do programa disponibiliza o comando `ollama` e, normalmente, uma API HTTP local em `http://127.0.0.1:11434`. A biblioteca Python faz chamadas a esse serviço.

9.1 Passo M1 — Baixar os modelos de referência

Em um terminal comum, Windows ou GNU/Linux:

```
ollama pull qwen3:1.7b
ollama pull embeddinggemma
ollama list
```

`qwen3:1.7b` é um modelo pequeno adequado a demonstrar respostas em português, sem a expectativa de comportamento comparável ao de modelos muito maiores. `embeddinggemma` gera vetores numéricos a partir de textos. No catálogo do Ollama, o primeiro ocupa aproximadamente 1,4 GB de download e o segundo aproximadamente 622 MB; esses tamanhos não correspondem necessariamente à memória total usada durante a inferência.

9.2 Passo M2 — Testar o modelo de linguagem

```
ollama run qwen3:1.7b
```

Quando o terminal permitir digitação, informe:

```
Explique em duas frases o que e recuperacao de informacao.
```

O modelo deve produzir texto. O teste demonstra a execução de um LLM, mas **não comprova** a construção de um RAG nem a correção factual da resposta. Para encerrar o modo interativo, utilize `/bye`.

9.3 Passo M3 — Testar geração e embeddings por Python

O script `scripts/testar_modelos.py` realiza uma chamada de chat e outra de embeddings usando a biblioteca Python. Execute:

Windows:

```
.\.venv\Scripts\python.exe scripts\testar_modelos.py
```

GNU/Linux:

```
./.venv/bin/python scripts/testar_modelos.py
```

Resultados esperados: uma resposta textual do modelo Qwen3, seguida de uma confirmação de que dois vetores foram gerados e a dimensão de cada vetor. A dimensão deve ser obtida do retorno efetivo da API; não é preciso fixar manualmente esse valor para testar a instalação.

Atenção: segurança ou configuração

Os modelos pequenos podem cometer erros e produzir afirmações sem base documental. Nesta fase não são fornecidos textos recuperados, portanto não devem ser avaliados como um assistente de consulta institucional. A distinção entre *geração de texto* e *geração fundamentada por recuperação* será central no experimento subsequente.

10 Verificar Streamlit e a interface web

O projeto contém `app_teste.py`, uma interface mínima cuja finalidade é confirmar que o Streamlit abre o navegador e responde a uma interação. Ela **não consulta Elasticsearch nem Ollama**. Isso é intencional: simplifica o diagnóstico, verificando a camada de interface de forma independente.

Windows:

```
.\.venv\Scripts\python.exe -m streamlit run app_teste.py
```

GNU/Linux:

```
./.venv/bin/python -m streamlit run app_teste.py
```

Abra http://127.0.0.1:8501 caso o navegador não inicie automaticamente. Digite uma pergunta e clique no botão de verificação. Para encerrar a aplicação, pressione **Ctrl+C** no terminal onde ela está sendo executada.

11 Procedimentos de encerramento e retomada

Um laboratório compartilhado precisa ter um ciclo claro de uso. Parar contêineres evita uso desnecessário de CPU/RAM; preservar os volumes conserva os índices para aulas futuras.

11.1 Parar os serviços ao fim da aula

Windows:

```
docker compose --profile ui down
```

GNU/Linux:

```
sudo docker compose --profile ui down
```

Esse comando remove os contêineres e a rede do projeto, mas **preserva os volumes nomeados**. Para retomar, execute `docker compose -profile ui up -d` (com `sudo` no GNU/Linux, quando necessário). A conta do Kibana e os dados do Elasticsearch permanecem no volume do serviço.

11.2 Reiniciar o modelo ou liberar memória

O serviço Ollama poderá manter um modelo carregado por algum tempo. Para verificar os modelos em execução:

```
ollama ps
```

Para descarregar um modelo específico, quando suportado pela versão instalada:

```
ollama stop qwen3:1.7b
```

Atenção: segurança ou configuração

Não use `docker compose down -v` por rotina. A opção `-v` também remove os volumes do Compose e pode apagar os índices e outros dados persistentes do laboratório. Em contextos de pesquisa, backups ou cópias dos documentos originais devem ser planejados antes de procedimentos destrutivos.

12 Segurança, permissões e boas práticas

A administração segura do laboratório também é conteúdo de aprendizagem. Ao longo deste roteiro, o estudante deve reconhecer a diferença entre **(a) privilégios de instalação**, **(b) permissões de execução** e **(c) credenciais de aplicação**.

Tema	Procedimento adotado	O que evitar
PowerShell	Usar `venv` sem ativação ou `RemoteSigned` apenas na sessão permitida.	Desabilitar políticas globais ou executar como administrador sem necessidade.
Python	Instalar pacotes no ambiente `.venv`.	`sudo pip install` ou instalações globais sem controle.
Docker Linux	Administrador executa Compose com `sudo` conforme política local.	Alterar `docker.sock` para permissões abertas ou distribuir grupo `docker` indiscriminadamente.
Elasticsearch	Autenticação habilitada, senhas locais, portas restritas ao loopback.	Desabilitar autenticação ou abrir a porta 9200 para a rede externa.
Kibana	Usar `kibana_system` somente para comunicação técnica e `elastic` para login de laboratório.	Publicar senhas ou usá-las em material compartilhado.
Ollama	Usar serviço local; registrar modelos instalados.	Habilitar acesso público à API sem análise de segurança.
Dados	Trabalhar com documentos públicos ou autorizados.	Indexar dados pessoais, confidenciais ou de terceiros sem autorização.
Reprodutibilidade	Guardar versões, logs sem segredos e descrição da máquina.	Publicar `.env`, chaves, tokens e capturas de tela com credenciais.

A configuração deste material deliberadamente não habilita TLS entre o navegador e o Kibana nem entre o cliente e o Elasticsearch. Isso é aceitável somente porque as portas foram associadas ao endereço de loopback e se trata de **exercício local em máquina controlada**; para conexões remotas ou serviços compartilhados pela rede, será necessário projetar autenticação, autorização, TLS, armazenamento seguro de segredos e controles de acesso específicos. Mesmo o endereço loopback pode ser acessado por outros processos executados no mesmo computador, razão pela qual credenciais e contas de usuário continuam relevantes.

13 Soluções para problemas frequentes

Sintoma	Causa provável	Diagnóstico / correção
`py` não reconhecido (Windows)	Python não instalado, launcher ausente ou terminal não reiniciado.	Verifique `python -version`; reinstale/configure Python pelo instalador oficial.

Sintoma	Causa provável	Diagnóstico / correção
Erro `unrecognized arguments: -m venv`	Dois comandos foram concatenados.	Execute apenas `py -m venv .venv` no Windows ou `python3 -m venv .venv` no Linux.
`Activate.ps1` bloqueado	Política de execução do PowerShell.	Verifique `Get-ExecutionPolicy -List`; use diretamente `.venv\Scripts \python .exe` se não puder ativar.
`ensurepip is not available`	Pacote `python3-venv` ausente no GNU/Linux.	Instale `python3-venv` com APT e recrie `.venv`.
`externally-managed-environment`	`pip` foi executado no Python gerenciado pelo sistema.	Ative o `venv` ou use `./.venv/bin/python -m pip`.
`docker` indisponível	Instalação incompleta ou sessão antiga.	Reabra o terminal; verifique Docker Desktop ou `systemctl status docker`.
`docker` permission denied	Usuário sem permissão no socket (Linux).	Use `sudo docker ...` conforme política; não altere permissões globais do socket.
Docker Desktop não inicia	WSL/virtualização ausentes ou bloqueados.	Consulte `wsl -status`, `wsl -update` e configuração de virtualização no firmware.
`vm.max_map_count` muito baixo	Parâmetro do kernel incompatível com Elasticsearch.	Aplicar `1048576` por procedimento autorizado.
Elasticsearch reinicia	Falta de memória, configuração do kernel ou senha/volume inconsistente.	`docker compose logs -tail=100 elasticsearch`; revisar RAM, `vm.max_map_count` e `.env`.
`401 Unauthorized` no Elasticsearch	Senha incorreta; senha de volume antigo.	Conferir `.env` e se os volumes são da mesma instalação. Nunca mostrar a senha em logs.
Kibana não conecta	Senha `kibana_system` ainda não configurada.	Execute `scripts/configurar_kibana.py`, confira o estado do Elasticsearch, reinicie Kibana.
Porta 9200/5601 ocupada	Outro serviço/contêiner escuta a mesma porta.	`docker ps`; identifique o processo antes de mudar as portas do projeto.
`ollama` não reconhecido	Instalador ausente ou terminal antigo.	Verifique instalação, reabra terminal; no Linux cheque `systemctl status ollama`.
Conexão recusada 11434	Serviço Ollama não iniciado.	Inicie o aplicativo/serviço e repita `ollama list`.
Modelo não encontrado	Download ainda não foi feito.	`ollama pull qwen3:1.7b` ou `ollama pull embeddinggemma`.
`ModuleNotFoundError`	Biblioteca não instalada no Python utilizado.	Execute `python -m pip list` com o Python de `.venv`; selecione o interpretador correto no VS Code.

Sintoma	Causa provável	Diagnóstico / correção
Streamlit não abre	Porta 8501 ocupada, Python incorreto ou falha da aplicação.	Verifique mensagem no terminal; execute `python -m streamlit run app_teste.py`.
Modelo muito lento	Inferência realizada em CPU, contexto longo ou outros processos consumindo RAM.	Use `qwen3:1.7b`, feche serviços dispensáveis e limite o tamanho das solicitações.

14 Checklist de preparação e relatório de evidências

Este checklist pode ser utilizado como formulário de conferência da estação. Marque os itens concluídos e acrescente as evidências solicitadas no relatório da atividade.

14.1 Checklist de infraestrutura

- ❑ Sistema operacional identificado.
- ❑ CPU, RAM e espaço em disco conferidos.
- ❑ Python 3 instalado e versão registrada.
- ❑ VS Code e extensão Python instalados.
- ❑ WSL 2 funcional (somente Windows).
- ❑ Docker e Compose testados.
- ❑ `vm.max_map_count` adequado.
- ❑ Ollama disponível.
- ❑ `.venv` criado.
- ❑ Bibliotecas de `requirements.txt` instaladas.
- ❑ `.env` gerado e protegido.
- ❑ Elasticsearch inicializado.
- ❑ Kibana conectado e acessível.
- ❑ Modelo Qwen3 baixado e testado.
- ❑ EmbeddingGemma baixado e testado.
- ❑ Aplicação Streamlit aberta.
- ❑ Versões registradas.
- ❑ Contêineres encerrados sem apagar volumes.

14.2 Evidências para entrega ao docente

1. Identificação da estação: distribuição/edição do sistema operacional, versão do Python, quantidade de RAM, espaço livre e arquitetura.
2. Resultado de `docker compose ps` e, se aplicável, captura da interface do Kibana autenticada, **sem revelar senhas**.
3. Execução de `scripts/validar_ambiente.py`, incluindo estado dos serviços e bibliotecas.
4. Execução de `scripts/testar_modelos.py`, com resposta textual e dimensão do embedding.
5. Captura da página `http://127.0.0.1:8501` contendo a interface de teste do Streamlit.
6. Arquivo `versoes-instaladas.txt`, produzido a partir do ambiente virtual da estação.
7. Relato de eventuais dificuldades, comandos utilizados para diagnóstico e sua resolução.

15 Atividades de reflexão e avaliação

A preparação do ambiente não deve ser considerada apenas um processo mecânico. Responda às questões a seguir com base na observação do computador utilizado.

1. Por que é recomendável criar um `.venv` para cada projeto Python? Como podemos comprovar qual interpretador está ativo?
2. Qual é a diferença entre a instalação do Ollama e a instalação da biblioteca Python `ollama`?
3. Por que o serviço Elasticsearch precisa armazenar dados em volumes persistentes?
4. O que significa publicar um serviço em `127.0.0.1`? Em que situação essa configuração seria insuficiente?
5. Qual é o risco de adicionar indiscriminadamente estudantes ao grupo `docker` no GNU/Linux?
6. Por que uma resposta gerada por `qwen3:1.7b`, sem documentos recuperados, ainda não constitui um RAG?
7. Qual é a diferença entre vetores produzidos por `embeddinggemma` e respostas em linguagem natural geradas por Qwen3?
8. Como o docente pode avaliar se duas estações estão reproduzindo o mesmo experimento, mesmo que seus resultados textuais não sejam idênticos?

16 Próxima etapa da disciplina

Quando a infraestrutura estiver validada, iniciaremos o estudo da recuperação documental. A proposta inicial é utilizar um conjunto de documentos institucionais públicos, como o Relatório de Gestão do IFPR de 2023, e comparar **BM25, busca vetorial e recuperação híbrida**. O projeto poderá evoluir para perguntas com respostas fundamentadas e referências de páginas.

A ordem didática sugerida é: (1) selecionar documentos e explicar direitos de uso; (2) extrair e inspecionar texto; (3) produzir fragmentos e metadados; (4) indexar no Elasticsearch; (5) gerar embeddings; (6) medir relevância; (7) fornecer trechos ao modelo local; (8) avaliar fidelidade, citações e ausência de evidência. O presente manual cobre exclusivamente a preparação dessa infraestrutura.

A Arquivo Compose de referência (`compose.yaml`)

O conteúdo a seguir permite auditar a configuração fornecida no pacote. As imagens Elastic são fixadas na série 9.5.6 para continuidade didática; os segredos são lidos de `.env` e não são publicados no código.

```
name: rag-local-didatico

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:${STACK_VERSION:-9.5.6}
    container_name: rag-local-elasticsearch
    environment:
      discovery.type: single-node
      xpack.security.enabled: "true"
      xpack.security.http.ssl.enabled: "false"
      xpack.security.transport.ssl.enabled: "false"
      xpack.ml.enabled: "false"
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD:?Execute scripts/criar_env.py antes}
      ES_JAVA_OPTS: -Xms1g -Xmx1g
    ports:
      - "127.0.0.1:9200:9200"
    volumes:
      - esdata:/usr/share/elasticsearch/data
    mem_limit: 3g
    healthcheck:
      test: ["CMD-SHELL", "curl -s -o /dev/null -w '%{http_code}' http://localhost:9200 | grep
     -q '^401$'"]
      interval: 15s
      timeout: 10s
      retries: 30
      start_period: 60s
    restart: unless-stopped

  kibana:
    image: docker.elastic.co/kibana/kibana:${STACK_VERSION:-9.5.6}
    container_name: rag-local-kibana
    profiles: ["ui"]
    depends_on:
      elasticsearch:
        condition: service_healthy
    environment:
      SERVER_NAME: rag-local-kibana
      ELASTICSEARCH_HOSTS: http://elasticsearch:9200
      ELASTICSEARCH_USERNAME: kibana_system
      ELASTICSEARCH_PASSWORD: ${KIBANA_SYSTEM_PASSWORD:?Execute scripts/criar_env.py antes}
      XPACK_SECURITY_ENCRYPTIONKEY: ${KIBANA_ENCRYPTION_KEY:?Execute scripts/criar_env.py
    antes}
    ports:
      - "127.0.0.1:5601:5601"
    volumes:
      - kibanadata:/usr/share/kibana/data
    mem_limit: 2g
    restart: unless-stopped

volumes:
  esdata:
  kibanadata:
```

B Referências técnicas oficiais

As fontes seguintes são adequadas para aprofundar comandos, requisitos mínimos, políticas de segurança e versões. Foram consultadas na preparação desta edição; páginas de instalação podem ser atualizadas pelos fornecedores.

1. PYTHON SOFTWARE FOUNDATION. *venv — Creation of virtual environments*. https://docs.python.org/3/library/venv.html.
2. PYTHON PACKAGING AUTHORITY. *Installing packages using pip and virtual environments*. https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/.
3. MICROSOFT. *about_Execution_Policies*. https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies.
4. MICROSOFT. *Install WSL*. https://learn.microsoft.com/windows/wsl/install.
5. DOCKER. *Install Docker Desktop on Windows*. https://docs.docker.com/desktop/setup/install/windows-install/.
6. DOCKER. *Install Docker Engine on Ubuntu*. https://docs.docker.com/engine/install/ubuntu/.
7. DOCKER. *Install Docker Engine on Debian*. https://docs.docker.com/engine/install/debian/.
8. DOCKER. *Linux post-installation steps for Docker Engine*. https://docs.docker.com/engine/install/linux-postinstall/.
9. ELASTIC. *Start a single-node Elasticsearch cluster in Docker*. https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-docker-basic.
10. ELASTIC. *Configure Elasticsearch Docker images in production*. https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-elasticsearch-docker-prod.
11. ELASTIC. *Install Kibana with Docker*. https://www.elastic.co/docs/deploy-manage/deploy/self-managed/install-kibana-with-docker.
12. OLLAMA. *Windows*. https://docs.ollama.com/windows.
13. OLLAMA. *Linux*. https://docs.ollama.com/linux.
14. OLLAMA. *Qwen3:1.7b*. https://ollama.com/library/qwen3:1.7b.
15. OLLAMA. *EmbeddingGemma*. https://ollama.com/library/embeddinggemma.
16. STREAMLIT. *Get started*. https://docs.streamlit.io/get-started.

Fim do manual de preparação do ambiente.

A próxima aula poderá começar com uma pergunta documental, um índice de textos e a comparação de estratégias de recuperação.